Practica 1. Base de datos distribuidas

2026/2 3TM2 UPIITA – IPN

Integrantes:

Gutiérrez Flores José Julián

Muñoz Reyes Citlali

Esta práctica utiliza la base de datos AdventureWorks de SQL Server para aplicar consultas multitablas, subconsultas, GROUP BY y conceptos avanzados. Los ejercicios requieren análisis de múltiples tablas como Sales.SalesOrderHeader, Sales.SalesOrderDetail, Production.Product, HumanResources.Employee y Person.Person.

Requisitos:

- Instala AdventureWorks desde el sitio oficial de Microsoft (versión OLTP).
- Usa SQL Server Management Studio (SSMS) conectado a la base de datos.
- Para cada consulta añade en el reporte el script de la consulta con discusión de la solución y captura de las 10 primeras filas del resultado.
- Subir al repositorio de cada alumno, reporte escrito de la práctica en el archivo practica1_equipoX.pdf y practica1_equipoX.SQL con scripts de las consultas.

Ejercicio 1. Encuentra los 10 productos más vendidos en 2014, mostrando nombre del producto, cantidad total vendida y nombre del cliente.

Practica 1 B...022 (sa (52))

```sql
WITH Top10Productos AS (
    --10 productos más vendidos del 2014
    SELECT TOP 10
        sod.ProductID,
        SUM(sod.OrderQty) AS CantidadTotalGlobal
    FROM Sales.SalesOrderDetail sod
    JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
    WHERE YEAR(soh.OrderDate) = 2014
    GROUP BY sod.ProductID
    ORDER BY CantidadTotalGlobal DESC
),
VentasDetalladas AS (
    -- Cantidad que compro cada cliente de esos 10 productos
    SELECT
        t10.ProductID,
        t10.CantidadTotalGlobal,
        soh.CustomerID,
        SUM(sod.OrderQty) AS CantidadPorCliente,
        ROW_NUMBER() OVER(PARTITION BY t10.ProductID ORDER BY SUM(sod.OrderQty) DESC) as RankingCliente
    FROM Top10Productos t10
    JOIN Sales.SalesOrderDetail sod ON t10.ProductID = sod.ProductID
    JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
    WHERE YEAR(soh.OrderDate) = 2014
    GROUP BY t10.ProductID, t10.CantidadTotalGlobal, soh.CustomerID
)
--Union de nombre de productos y personas
SELECT
    p.Name AS [Nombre del Producto],
    vd.CantidadTotalGlobal AS [Total Vendido 2014],
    per.FirstName + ' ' + per.LastName AS [Cliente que más compró],
    vd.CantidadPorCliente AS [Cantidad comprada por este cliente]
FROM VentasDetalladas vd
JOIN Production.Product p ON vd.ProductID = p.ProductID
JOIN Sales.Customer c ON vd.CustomerID = c.CustomerID
JOIN Person.Person per ON c.PersonID = per.BusinessEntityID
WHERE vd.RankingCliente = 1
ORDER BY vd.CantidadTotalGlobal DESC;
```

Para la solución de este ejercicio se hizo uso de comandos como lo son JOIN, GROUP BY, SUM(), TOP, ORDER BY, CTE (WITH), ROW_NUMBER()←Nuevo comando diferente a los que hemos ocupado en clase

Esto con la finalidad de hacer la consulta más legible y modular, ya que permite que se pueda ver la consulta dividida por pasos.

Respuesta:

```sql
/*Ejercicio 1. Encuentra los 10 productos más vendidos en 2014, mostrando nombre
del producto, cantidad total vendida y nombre del cliente.*/

WITH Top10Productos AS (
    --10 productos más vendidos del 2014
    SELECT TOP 10
        sod.ProductID,
        SUM(sod.OrderQty) AS CantidadTotalGlobal
    FROM Sales.SalesOrderDetail sod
    JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
    WHERE YEAR(soh.OrderDate) = 2014
    GROUP BY sod.ProductID
    ORDER BY CantidadTotalGlobal DESC
),
```

100 % 10 0

Resultados | Mensajes

	Nombre del Producto	Total Vendido 2014	Cliente que más compró	Cantidad comprada por este cliente
1	Water Bottle - 30 oz.	2902	Alexander Berger	24
2	AWC Logo Cap	1950	Pilar Ackerman	25
3	Sport-100 Helmet, Blue	1776	Kevin Liu	21
4	Patch Kit/8 Patches	1761	Jon Grande	11
5	Sport-100 Helmet, Red	1717	Chris Bidelman	19
6	Sport-100 Helmet, Black	1716	Richard Lum	23
7	Mountain Tire Tube	1501	Nancy Chapman	4
8	Classic Vest, S	1395	Pilar Ackerman	39
9	Short-Sleeve Classic Jersey, XL	1264	Min Su	32
10	Road Tire Tube	1156	Jennifer Simmons	6

1. Una vez resuelta la consulta: agrega el precio unitario promedio (AVG(UnitPrice)) y filtra solo productos con ListPrice > 1000.

```sql
--Solucion Practica 1 Ejercicio 1.1 Una vez resuelta la consulta: agrega el precio unitario
--promedio (AVG(UnitPrice)) y filtra solo productos con ListPrice > 1000.
SELECT
    p.Name,
    t.cant,
    t.PromedioPrecio,
    MAX(per.FirstName + ' ' + per.LastName) AS [Cliente (Ejemplo)]
FROM Production.Product p
JOIN (
    SELECT TOP 10
        sod.ProductID,
        SUM(sod.OrderQty) AS cant,
        AVG(sod.UnitPrice) AS PromedioPrecio
    FROM sales.SalesOrderDetail sod
    JOIN sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID
    JOIN Production.Product p_f ON sod.ProductID = p_f.ProductID
    WHERE YEAR(soh.OrderDate) = '2014'
      AND p_f.ListPrice > 1000
    GROUP BY sod.ProductID
    ORDER BY cant DESC
) AS t ON p.ProductID = t.ProductID
JOIN sales.SalesOrderDetail sod_all ON p.ProductID = sod_all.ProductID
JOIN sales.SalesOrderHeader soh_all ON sod_all.SalesOrderID = soh_all.SalesOrderID
JOIN sales.Customer c ON soh_all.CustomerID = c.CustomerID
JOIN Person.Person per ON c.PersonID = per.BusinessEntityID
WHERE YEAR(soh_all.OrderDate) = '2014'
GROUP BY p.Name, t.cant, t.PromedioPrecio
ORDER BY t.cant DESC;
```

Esta implementación se ocupó AVG(UnitPrice), ListPrice (para filtrar) y la subconsulta JOIN, con el fin de permitir que se redujera el volumen de datos y obtener el precio representativo evitando sesgos por valores extremos

Resultado:

```sql
--Solucion Practica 1 Ejercicio 1.1 Una vez resuelta la consulta: agrega el precio unitario
--promedio (AVG(UnitPrice)) y filtra solo productos con ListPrice > 1000.
SELECT
    p.Name,
    t.cant,
    t.PromedioPrecio,
    MAX(per.FirstName + ' ' + per.LastName) AS [Cliente (Ejemplo)]
FROM Production.Product p
JOIN (
    SELECT TOP 10
```

100 % 10 0 Línea: 59, Carácte

Resultados Mensajes

	Name	cant	PromedioPrecio	Cliente (Ejemplo)
1	Mountain-200 Black, 38	619	1981.7872	Yao-Qiang Cheng
2	Road-350-W Yellow, 48	586	1388.3549	Wyatt Green
3	Road-350-W Yellow, 40	530	1391.601	Yuhong Li
4	Mountain-200 Black, 42	513	2058.144	Zachary Robins...
5	Mountain-200 Silver, 38	508	2079.2043	Yan Li
6	Mountain-200 Silver, 42	441	2086.7745	Zachary Ross
7	Mountain-200 Black, 46	437	2050.3074	Zachary Chen
8	Mountain-200 Silver, 46	432	2095.4693	Yale Li
9	Touring-1000 Yellow, ...	397	1812.1551	Wyatt Martinez
10	Touring-1000 Blue, 60	393	1839.7073	Wayne Tang

2. Documenta la solución inicial y solución con las variantes solicitadas.

La solución inicial tiene un enfoque para poder obtener los 10 productos más vendidos en determinado año, siendo así que se obtengan de manera correcta los nombres de los productos y cantidad vendida, sin embargo, a comparación de la implementación esta consulta puede ser difícil de mantener conforme se agreguen más detalles.

En la solución de implementación se opta por incorporar CTE que es un comando para creación de tabla temporal y la función de ROW_NUMBER(), lo cual hace que el problema se pueda apreciar por etapas y especificar los filtros que necesitaba.

Conclusión:

En este caso preferimos el uso de CTE porque hace más legible, modular, y escalable la consulta, siendo así que se reutilicen los datos sin la necesidad de estar calculándolos de nuevo asi como se hace en la primera parte

Ejercicio 2: Lista los empleados que han vendido más que el promedio de ventas por empleado en el territorio 'Northwest'.

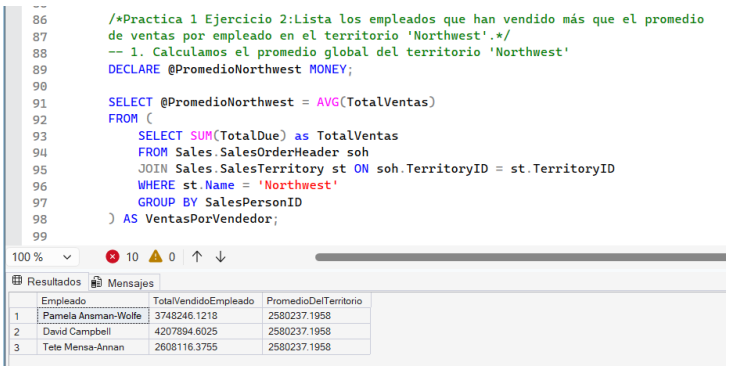
```sql
/*Practica 1 Ejercicio 2:Lista los empleados que han vendido más que el promedio
de ventas por empleado en el territorio 'Northwest'.*/
-- 1. Calculamos el promedio global del territorio 'Northwest'
DECLARE @PromedioNorthwest MONEY;

SELECT @PromedioNorthwest = AVG(TotalVentas)
FROM (
    SELECT SUM(TotalDue) as TotalVentas
    FROM Sales.SalesOrderHeader soh
    JOIN Sales.SalesTerritory st ON soh.TerritoryID = st.TerritoryID
    WHERE st.Name = 'Northwest'
    GROUP BY SalesPersonID
) AS VentasPorVendedor;

-- 2. Listamos a los empleados que superan ese promedio
SELECT
    p.FirstName + ' ' + p.LastName AS Empleado,
    SUM(soh.TotalDue) AS TotalVendidoEmpleado,
    @PromedioNorthwest AS PromedioDelTerritorio
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesTerritory st ON soh.TerritoryID = st.TerritoryID
JOIN Sales.SalesPerson sp ON soh.SalesPersonID = sp.BusinessEntityID
JOIN Person.Person p ON sp.BusinessEntityID = p.BusinessEntityID
WHERE st.Name = 'Northwest'
GROUP BY p.FirstName, p.LastName, sp.BusinessEntityID
HAVING SUM(soh.TotalDue) > @PromedioNorthwest;
```

Se ocupó Declare, la subconsulta JOIN de manera “anidada” para poder evitar que se repitieran cálculos, mejorar el rendimiento y que se vea más clara;

Resultado:



```sql
/*Practica 1 Ejercicio 2:Lista los empleados que han vendido más que el promedio
de ventas por empleado en el territorio 'Northwest'.*/
-- 1. Calculamos el promedio global del territorio 'Northwest'
DECLARE @PromedioNorthwest MONEY;

SELECT @PromedioNorthwest = AVG(TotalVentas)
FROM (
    SELECT SUM(TotalDue) as TotalVentas
    FROM Sales.SalesOrderHeader soh
    JOIN Sales.SalesTerritory st ON soh.TerritoryID = st.TerritoryID
    WHERE st.Name = 'Northwest'
    GROUP BY SalesPersonID
) AS VentasPorVendedor;
```

100 % 10 0

Resultados | Mensajes

	Empleado	TotalVendidoEmpleado	PromedioDelTerritorio
1	Pamela Ansman-Wolfe	3748246.1218	2580237.1958
2	David Campbell	4207894.6025	2580237.1958
3	Tete Mensa-Annan	2608116.3755	2580237.1958

1. Requisito adicional: aplicar subconsultas.

```sql
---1.   Requisito adicional: aplicar subconsultas.
--Solucion Usando Subconsultas
SELECT
    p.FirstName + ' ' + p.LastName AS Empleado,
    SUM(soh.TotalDue) AS TotalVendidoEmpleado
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesPerson sp ON soh.SalesPersonID = sp.BusinessEntityID
JOIN Person.Person p ON sp.BusinessEntityID = p.BusinessEntityID
JOIN Sales.SalesTerritory st ON soh.TerritoryID = st.TerritoryID
WHERE st.Name = 'Northwest'
GROUP BY p.FirstName, p.LastName
HAVING SUM(soh.TotalDue) > (
    -- Subconsulta: El promedio de Northwest
    SELECT AVG(VentasTotales)
    FROM (
        SELECT SUM(TotalDue) AS VentasTotales
        FROM Sales.SalesOrderHeader soh2
        JOIN Sales.SalesTerritory st2 ON soh2.TerritoryID = st2.TerritoryID
        WHERE st2.Name = 'Northwest'
        GROUP BY SalesPersonID
    ) AS Sub
);
```

Se añadió subconsultas las cuales ayudaron a comparar contra un valor calculado de manera “dinámica”

Resultado:

```sql
---1.   Requisito adicional: aplicar subconsultas.
--Solucion Usando Subconsultas
SELECT
    p.FirstName + ' ' + p.LastName AS Empleado,
    SUM(soh.TotalDue) AS TotalVendidoEmpleado
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesPerson sp ON soh.SalesPersonID = sp.BusinessEntityID
JOIN Person.Person p ON sp.BusinessEntityID = p.BusinessEntityID
JOIN Sales.SalesTerritory st ON soh.TerritoryID = st.TerritoryID
WHERE st.Name = 'Northwest'
GROUP BY p.FirstName, p.LastName
HAVING SUM(soh.TotalDue) > (
    -- Subconsulta: El promedio de Northwest
    SELECT AVG(VentasTotales)
```

100 % 10 0

Resultados | Mensajes

	Empleado	TotalVendidoEmpleado
1	Pamela Ansman-Wolfe	3748246.1218
2	David Campbell	4207894.6025
3	Tete Mensa-Annan	2608116.3755

2. Una vez resuelta la consulta convierte la subconsulta en un CTE (Common Table Expresión).

```sql
---2.   Una vez resuelta la consulta convierte la subconsulta en un CTE (Common Table Expresión).
--Conversion a CTE
WITH VentasVendedores AS (
    SELECT
        p.FirstName + ' ' + p.LastName AS Empleado,
        SUM(soh.TotalDue) AS TotalVendido
    FROM Sales.SalesOrderHeader soh
    JOIN Sales.SalesPerson sp ON soh.SalesPersonID = sp.BusinessEntityID
    JOIN Person.Person p ON sp.BusinessEntityID = p.BusinessEntityID
    JOIN Sales.SalesTerritory st ON soh.TerritoryID = st.TerritoryID
    WHERE st.Name = 'Northwest'
    GROUP BY p.FirstName, p.LastName
),
PromedioFinal AS (
    SELECT AVG(TotalVendido) AS PromedioGlobal FROM VentasVendedores
)
SELECT
    v.Empleado,
    v.TotalVendido AS TotalVendidoEmpleado,
    p.PromedioGlobal AS PromedioDelTerritorio
FROM VentasVendedores v, PromedioFinal p
WHERE v.TotalVendido > p.PromedioGlobal;
```

Se hizo uso de CTE para mejor la consulta y permitir reutilizar datos y facilitar el escalamiento de la consulta

Resultado:

```sql
---2.   Una vez resuelta la consulta convierte la subconsulta en un CTE (Common Table Expresión).
--Conversion a CTE
WITH VentasVendedores AS (
    SELECT
        p.FirstName + ' ' + p.LastName AS Empleado,
        SUM(soh.TotalDue) AS TotalVendido
    FROM Sales.SalesOrderHeader soh
    JOIN Sales.SalesPerson sp ON soh.SalesPersonID = sp.BusinessEntityID
    JOIN Person.Person p ON sp.BusinessEntityID = p.BusinessEntityID
    JOIN Sales.SalesTerritory st ON soh.TerritoryID = st.TerritoryID
    WHERE st.Name = 'Northwest'
    GROUP BY p.FirstName, p.LastName
```

100 % 10 0 Línea: 155, Carácter: 41

Resultados | Mensajes

	Empleado	TotalVendidoEmpleado	PromedioDelTerritorio
1	Pamela Ansman-Wolfe	3748246.1218	2338092.9534
2	David Campbell	4207894.6025	2338092.9534
3	Tete Mensa-Annan	2608116.3755	2338092.9534
4	Linda Mitchell	2353659.2774	2338092.9534

3. Documenta la solución inicial y solución con la variante solicitada.

Solución base: se apreció el uso de una variable DECLARE, esta con el fin de poder almacenar el promedio de ventas que permitió evitar cálculos repetidos y hacer las comparaciones, sin embargo, un punto malo es que depende del uso de variables, lo cual al largo puede presentar un problema.

Implementación 1: aquí se remplazó la parte de “uso de variables” y se hace la consulta de manera “dinámica”, puesto que estamos ocupando Having para hacer subconsultas.

Implementación 2: el uso de CTE a nuestro parecer ayuda más a que sea una consulta clara, reutilizando resultados y facilitando la lectura aparte del mantenimiento de la consulta

Conclusión: entre las tres aportaciones, CTE presenta mayor claridad ya que puede resolver las consultas si estas son con mayor complejidad

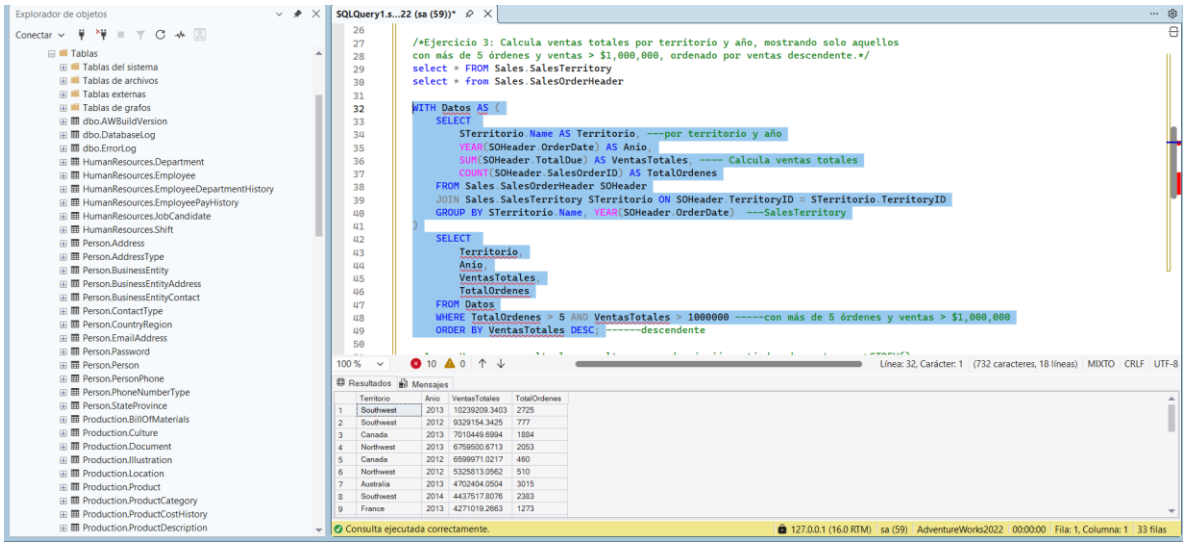
Ejercicio 3: Calcula ventas totales por territorio y año, mostrando solo aquellos con más de 5 órdenes y ventas > $1,000,000, ordenado por ventas descendente.

```sql
/*Ejercicio 3: Calcula ventas totales por territorio y año, mostrando solo aquellos
con más de 5 órdenes y ventas > $1,000,000, ordenado por ventas descendente.*/
select * FROM Sales.SalesTerritory
select * from Sales.SalesOrderHeader

WITH Datos AS (
    SELECT
        STerritorio.Name AS Territorio, ---por territorio y año
        YEAR(SOHeader.OrderDate) AS Anio,
        SUM(SOHeader.TotalDue) AS VentasTotales, ---- Calcula ventas totales
        COUNT(SOHeader.SalesOrderID) AS TotalOrdenes
    FROM Sales.SalesOrderHeader SOHeader
    JOIN Sales.SalesTerritory STerritorio ON SOHeader.TerritoryID = STerritorio.TerritoryID
    GROUP BY STerritorio.Name, YEAR(SOHeader.OrderDate)  ---SalesTerritory
)
    SELECT
        Territorio,
        Anio,
        VentasTotales,
        TotalOrdenes
    FROM Datos
    WHERE TotalOrdenes > 5 AND VentasTotales > 1000000 -----con más de 5 órdenes y ventas > $1,000,000
    ORDER BY VentasTotales DESC; ------descendente
```

Ocupamos para esta consulta el uso de Count, SUM, STDEV,CTE; esto para doder medir variables de ventas, permitir análisis estadísticos, mejorar la claridad de la consulta y la reutilización de datos.

Resultado:

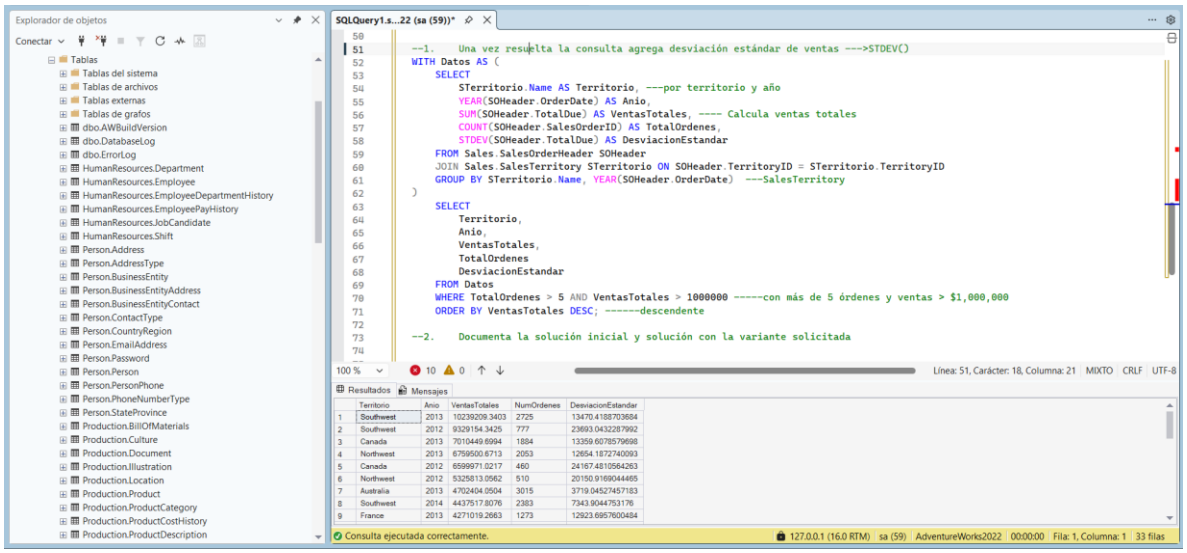


1. Una vez resuelta la consulta agrega desviación estándar de ventas

```sql
--1.    Una vez resuelta la consulta agrega desviación estándar de ventas --->STDEV()
WITH Datos AS (
    SELECT
        STerritorio.Name AS Territorio, ---por territorio y año
        YEAR(SOHeader.OrderDate) AS Anio,
        SUM(SOHeader.TotalDue) AS VentasTotales, ---- Calcula ventas totales
        COUNT(SOHeader.SalesOrderID) AS TotalOrdenes,
        STDEV(SOHeader.TotalDue) AS DesviacionEstandar
    FROM Sales.SalesOrderHeader SOHeader
    JOIN Sales.SalesTerritory STerritorio ON SOHeader.TerritoryID = STerritorio.TerritoryID
    GROUP BY STerritorio.Name, YEAR(SOHeader.OrderDate)  ---SalesTerritory
)
        SELECT *
    FROM Datos
    WHERE TotalOrdenes > 5 AND VentasTotales > 1000000 -----con más de 5 órdenes y ventas > $1,000,000
    ORDER BY VentasTotales DESC; ------descendente
```

En esencia solo se agregó la implementación de desviación estándar de ventas

Resultado:



2. Documenta la solución inicial y solución con la variante solicitada.

Solución base:

Se cumple con lo solicitado, de manera exacta.

Solución variante: se implementa la desviación estándar (STDEV), añadiendo otro tipo de “filtro” a la consulta

Conclusión: es casi la misma consulta, pero la variación de la implementación causa que la consulta sea más completa

Ejercicio 4: Encuentra vendedores que han vendido TODOS los productos de la categoría "Bikes".

```sql
/*Ejercicio 4: Encuentra vendedores que han vendido TODOS los productos de la categoría "Bikes".*/
use AdventureWorks2022
SELECT
    P.FirstName + ' ' + P.LastName AS Vendedor,
    COUNT(DISTINCT SOD.ProductID) AS ProductosBikesVendidos
FROM Sales.SalesOrderHeader SOH
JOIN Sales.SalesOrderDetail SOD ON SOH.SalesOrderID = SOD.SalesOrderID
JOIN Production.Product PR ON SOD.ProductID = PR.ProductID
JOIN Production.ProductSubcategory PS ON PR.ProductSubcategoryID = PS.ProductSubcategoryID
JOIN Production.ProductCategory PC ON PS.ProductCategoryID = PC.ProductCategoryID
JOIN HumanResources.Employee E ON SOH.SalesPersonID = E.BusinessEntityID
JOIN Person.Person P ON E.BusinessEntityID = P.BusinessEntityID
WHERE PC.Name = 'Bikes'
GROUP BY P.FirstName, P.LastName
HAVING COUNT(DISTINCT SOD.ProductID) = (
    -- Esta subconsulta cuenta cuántas "Bikes" existen en total
    SELECT COUNT(ProductID)
    FROM Production.Product P2
    JOIN Production.ProductSubcategory PS2 ON P2.ProductSubcategoryID = PS2.ProductSubcategoryID
    JOIN Production.ProductCategory PC2 ON PS2.ProductCategoryID = PC2.ProductCategoryID
    WHERE PC2.Name = 'Bikes'
);
```

Un comando ocupado que no se había ocupado en las consultas anteriores es el HAVING que en este caso se ocupó para hacer la parte de “filtrar”, garantizando evitar datos duplicados con el distinct

Resultado:

```sql
210 /*Ejercicio 4: Encuentra vendedores que han vendido TODOS los productos de la categoría "Bikes"
211 use AdventureWorks2022
212 SELECT
213     P.FirstName + ' ' + P.LastName AS Vendedor,
214     COUNT(DISTINCT SOD.ProductID) AS ProductosBikesVendidos
215 FROM Sales.SalesOrderHeader SOH
216 JOIN Sales.SalesOrderDetail SOD ON SOH.SalesOrderID = SOD.SalesOrderID
217 JOIN Production.Product PR ON SOD.ProductID = PR.ProductID
218 JOIN Production.ProductSubcategory PS ON PR.ProductSubcategoryID = PS.ProductSubcategoryID
219 JOIN Production.ProductCategory PC ON PS.ProductCategoryID = PC.ProductCategoryID
220 JOIN HumanResources.Employee E ON SOH.SalesPersonID = E.BusinessEntityID
221 JOIN Person.Person P ON E.BusinessEntityID = P.BusinessEntityID
222 WHERE PC.Name = 'Bikes'
223 GROUP BY P.FirstName, P.LastName
224 HAVING COUNT(DISTINCT SOD.ProductID) = (
225     -- Esta subconsulta cuenta cuántas "Bikes" existen en total
```

100 % 10 0

Resultados | Mensajes

	Vendedor	ProductosBikesVendidos
1	Jillian Carson	97
2	Shu Ito	97
3	Linda Mitchell	97
4	Tsvi Reiter	97
5	José Saraiva	97

Implementaciones:

1. Cambia a categoría "Clothing" (ID=4).

```sql
--1.    Cambia a categoría "Clothing" (ID=4).
SELECT
    P.FirstName + ' ' + P.LastName AS Vendedor,
    COUNT(DISTINCT SOD.ProductID) AS ProductosRopaVendidos
FROM Sales.SalesOrderHeader SOH
JOIN Sales.SalesOrderDetail SOD ON SOH.SalesOrderID = SOD.SalesOrderID
JOIN Production.Product PR ON SOD.ProductID = PR.ProductID
JOIN Production.ProductSubcategory PS ON PR.ProductSubcategoryID = PS.ProductSubcategoryID
JOIN HumanResources.Employee E ON SOH.SalesPersonID = E.BusinessEntityID
JOIN Person.Person P ON E.BusinessEntityID = P.BusinessEntityID
WHERE PS.ProductCategoryID = 4 -- Filtro directo por el ID de "Clothing"
GROUP BY P.FirstName, P.LastName
HAVING COUNT(DISTINCT SOD.ProductID) = (
    -- Cuenta cuántos productos existen en la categoría 4
    SELECT COUNT(ProductID)
    FROM Production.Product P2
    JOIN Production.ProductSubcategory PS2 ON P2.ProductSubcategoryID = PS2.ProductSubcategoryID
    WHERE PS2.ProductCategoryID = 4
);
```

Se implementó la parte solicitada, de forma que contara como un filtro

Resultado:

```sql
233 SELECT
234     P.FirstName + ' ' + P.LastName AS Vendedor,
235     COUNT(DISTINCT SOD.ProductID) AS ProductosRopaVendidos
236 FROM Sales.SalesOrderHeader SOH
237 JOIN Sales.SalesOrderDetail SOD ON SOH.SalesOrderID = SOD.SalesOrderID
238 JOIN Production.Product PR ON SOD.ProductID = PR.ProductID
239 JOIN Production.ProductSubcategory PS ON PR.ProductSubcategoryID = PS.ProductSub
240 JOIN HumanResources.Employee E ON SOH.SalesPersonID = E.BusinessEntityID
241 JOIN Person.Person P ON E.BusinessEntityID = P.BusinessEntityID
242 WHERE PS.ProductCategoryID = 4 -- Filtro directo por el ID de "Clothing"
243 GROUP BY P.FirstName, P.LastName
244 HAVING COUNT(DISTINCT SOD.ProductID) = (
245     -- Cuenta cuántos productos existen en la categoría 4
246     SELECT COUNT(ProductID)
247     FROM Production.Product P2
248     JOIN Production.ProductSubcategory PS2 ON P2.ProductSubcategoryID = PS2.Prod
249     WHERE PS2.ProductCategoryID = 4
250 );
251
252 --2.    Cuenta cuántos productos por categoría maneja cada vendedor.
253 SELECT
254     P.FirstName + ' ' + P.LastName AS Vendedor,
255     PC.Name AS Categoria,
```

100 % 10 0

Resultados | Mensajes

Vendedor	ProductosRopaVendidos

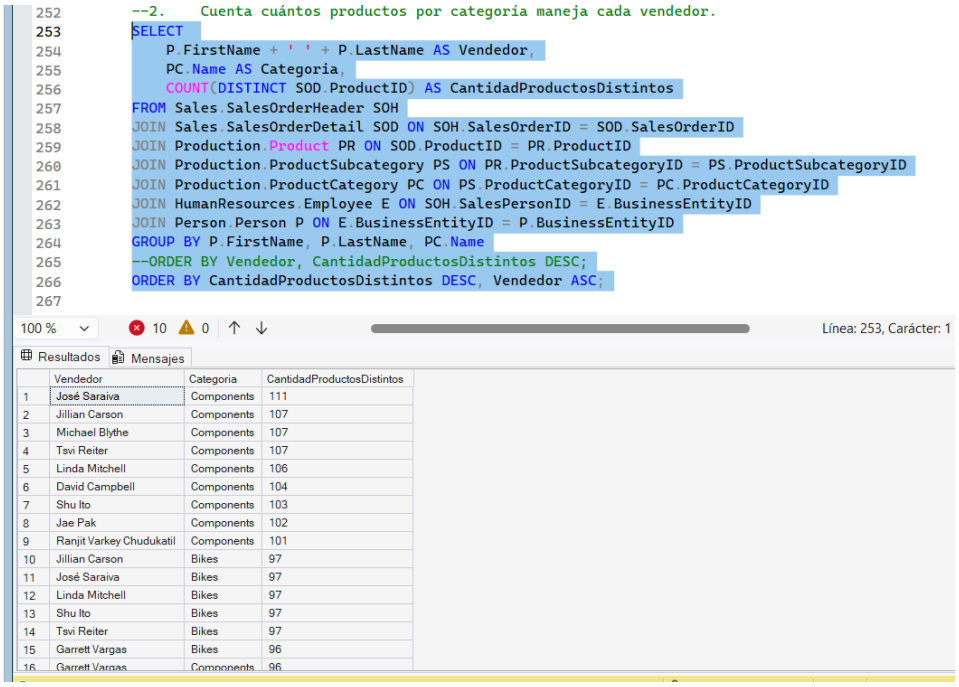
Sin embargo, no encontramos valores

2. Cuenta cuántos productos por categoría maneja cada vendedor.

```sql
--2.    Cuenta cuántos productos por categoría maneja cada vendedor.
SELECT
    P.FirstName + ' ' + P.LastName AS Vendedor,
    PC.Name AS Categoria,
    COUNT(DISTINCT SOD.ProductID) AS CantidadProductosDistintos
FROM Sales.SalesOrderHeader SOH
JOIN Sales.SalesOrderDetail SOD ON SOH.SalesOrderID = SOD.SalesOrderID
JOIN Production.Product PR ON SOD.ProductID = PR.ProductID
JOIN Production.ProductSubcategory PS ON PR.ProductSubcategoryID = PS.ProductSubcategoryID
JOIN Production.ProductCategory PC ON PS.ProductCategoryID = PC.ProductCategoryID
JOIN HumanResources.Employee E ON SOH.SalesPersonID = E.BusinessEntityID
JOIN Person.Person P ON E.BusinessEntityID = P.BusinessEntityID
GROUP BY P.FirstName, P.LastName, PC.Name
--ORDER BY Vendedor, CantidadProductosDistintos DESC;
ORDER BY CantidadProductosDistintos DESC, Vendedor ASC;
```

Se hizo uso de Group by para obtener los filtros solicitados.

Resultado:



100 % 10 0 Línea: 253, Carácter: 1

Resultados | Mensajes

	Vendedor	Categoria	CantidadProductosDistintos
1	José Saraiva	Components	111
2	Jillian Carson	Components	107
3	Michael Blythe	Components	107
4	Tsvi Reiter	Components	107
5	Linda Mitchell	Components	106
6	David Campbell	Components	104
7	Shu Ito	Components	103
8	Jae Pak	Components	102
9	Ranjit Varkey Chudukatil	Components	101
10	Jillian Carson	Bikes	97
11	José Saraiva	Bikes	97
12	Linda Mitchell	Bikes	97
13	Shu Ito	Bikes	97
14	Tsvi Reiter	Bikes	97
15	Garrett Vargas	Bikes	96
16	Garrett Vargas	Components	96

3. Documenta la solución inicial y solución con las variantes solicitadas.
   Conclusión: las implementaciones en esta consulta, hace que se amplié el análisis haciendo más robusta la consulta, en esencia la consulta se nota igual solo que con detalles más completos para obtener un mejor filtrado.

Ejercicio 5: Determinar el producto más vendido de cada categoría de producto, considerando el escenario de que el esquema SALES se encuentra en una instancia (servidor) A y el esquema PRODUCTION en otra instancia (servidor) B.

```sql
/* Ejercicio 5: Determinar el producto más vendido de cada categoría de producto,
considerando el escenario de que el esquema SALES se encuentra en una instancia (servidor) A
y el esquema PRODUCTION en otra instancia (servidor) B.*/

-- CTE 1: Ventas en el servidor local
WITH VentasLocales AS (
    SELECT ProductID, SUM(OrderQty) AS CantidadTotal
    FROM Sales.SalesOrderDetail
    GROUP BY ProductID
),
-- CTE 2: Catálogo en el servidor vinculado 'CITLALi'
CatalogoRemoto AS (
    SELECT
        p.ProductID,
        p.Name AS NombreProducto,
        c.Name AS NombreCategoria
    FROM CITLALi.AdventureWorks2022.Production.Product p
    JOIN CITLALi.AdventureWorks2022.Production.ProductSubcategory s ON p.ProductSubcategoryID = s.ProductSubcategoryID
    JOIN CITLALi.AdventureWorks2022.Production.ProductCategory c ON s.ProductCategoryID = c.ProductCategoryID
),
-- CTE 3: Identificar el producto más vendido por categoría
RankingProductos AS (
    SELECT
        cr.NombreCategoria,
        cr.NombreProducto,
        vl.CantidadTotal,
        RANK() OVER (PARTITION BY cr.NombreCategoria ORDER BY vl.CantidadTotal DESC) AS Posicion
    FROM VentasLocales vl
    JOIN CatalogoRemoto cr ON vl.ProductID = cr.ProductID
)
-- Consulta Final
SELECT NombreCategoria, NombreProducto, CantidadTotal
FROM RankingProductos
WHERE Posicion = 1
ORDER BY CantidadTotal DESC;
```

Y consulta con la propia máquina, desde la misma instancia

Practica 1 B...022 (sa (52))

```sql
--Con mi propia maquina
SELECT @@SERVERNAME;

-- Creamos el servidor vinculado usando tu nombre de instancia
EXEC sp_addlinkedserver
    @server = 'BOLILLOJULIAN\SQLEXPRESS',
    @srvproduct = '',
    @provider = 'SQLNCLI',
    @datasrc = 'BOLILLOJULIAN\SQLEXPRESS';

-- Configuramos para que use tus mismas credenciales actuales
EXEC sp_addlinkedsrvlogin
    @rmtsrvname = 'BOLILLOJULIAN\SQLEXPRESS',
    @useself = 'True';

----------
-- CTE 1: Agregamos las ventas desde el esquema SALES (Instancia A)
WITH VentasLocales AS (
    SELECT ProductID, SUM(OrderQty) AS CantidadTotal
    FROM Sales.SalesOrderDetail
    GROUP BY ProductID
),
-- CTE 2: Consultamos el catálogo desde el esquema PRODUCTION (Instancia B - Tu PC)
CatalogoRemoto AS (
    SELECT
        p.ProductID,
        p.Name AS NombreProducto,
        c.Name AS NombreCategoria
    -- Usamos el nombre de tu instancia: [Servidor].[Base].[Esquema].[Tabla]
    FROM [BOLILLOJULIAN\SQLEXPRESS].AdventureWorks2022.Production.Product p
    JOIN [BOLILLOJULIAN\SQLEXPRESS].AdventureWorks2022.Production.ProductSubcategory s
        ON p.ProductSubcategoryID = s.ProductSubcategoryID
    JOIN [BOLILLOJULIAN\SQLEXPRESS].AdventureWorks2022.Production.ProductCategory c
        ON s.ProductCategoryID = c.ProductCategoryID
),
```

Se ocupó Linked Servers para poder simular ambientes reales, permitiendo la consulta distribuida y evitara duplicar datos

En este caso, la parte de servidores vinculados nos faltó hacer demostración entre nuestros equipos, pero dejamos parte del código para poder hacerlo.